

Research Article

A New (Related-Key) Neural Distinguisher Using Two Differences for Differential Cryptanalysis

Gao Wang ¹, Gaoli Wang ^{1,2} and Siwei Sun ^{3,4}

¹Shanghai Key Laboratory of Trustworthy Computing, Software Engineering Institute, East China Normal University, Shanghai, China

²Advanced Cryptography and System Security Key Laboratory of Sichuan Province, Chengdu 610103, China

³School of Cryptology, University of Chinese Academy of Sciences, Beijing, China

⁴State Key Laboratory of Cryptology, Beijing 100878, China

Correspondence should be addressed to Gaoli Wang; glwang@sei.ecnu.edu.cn

Received 26 May 2024; Accepted 5 October 2024

Academic Editor: Qichun Wang

Copyright © 2024 Gao Wang et al. This is an open access article distributed under the Creative Commons Attribution License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

At CRYPTO 2019, Gohr showed the significant advantages of neural distinguishers over traditional distinguishers in differential cryptanalysis. At fast software encryption (FSE) 2024, Bellini et al. provided a generic tool to automatically train the (related-key) differential neural distinguishers for different block ciphers. In this paper, based on the intrinsic principle of differential cryptanalysis and neural distinguisher, we propose a superior (related-key) differential neural distinguisher that uses the ciphertext pairs generated by two different differences. In addition, we give a framework to automatically train our (related-key) differential neural distinguisher with four steps: difference selection, sample generation, training pipeline, and evaluation scheme. To demonstrate the effectiveness of our approach, we apply it to the block ciphers: Simon, Speck, Simeck, and Hight. Compared to the existing results, our method can provide improved accuracy and even increase the number of rounds that can be analyzed. The source codes are available in https://github.com/differentialedistinguisher/AutoND_New.

1. Introduction

Differential cryptanalysis [1] is one of the most effective techniques to evaluate the security of block ciphers. It distinguishes the block cipher from random permutations by analyzing the propagation properties of plaintext differences. In traditional differential cryptanalysis, cryptanalysts usually choose the differential path with the highest probability to perform this task. To search for the differential path with the highest probability, the most widely used methods are the branch-and-bound [2], Mixed Integer Linear Programming (MILP) [3], Constraint Programming (CP) [4], and Boolean Satisfiability problems or Satisfiability modulo theories (SAT/SMT) [5].

In recent years, with the continuous development of deep learning, cryptologists have begun to explore the neural network aided differential cryptanalysis technology. At Crypto 2019, Gohr [6] first introduced the concept of differential neural distinguisher and showed its superior performance over the traditional differential distinguisher for Speck with

block size 32 bits and key length 64 bits (Speck32/64) [7]. With the help of neural distinguishers in rounds 6, 7, and 8, he successfully reduced the computational complexity of 11th and 12th key recovery attacks against Speck32/64. Since then, this innovative approach has been applied to different block ciphers, such as Simon [7], Present [8] and GIMLI [9], and so on. In neural network aided differential cryptanalysis, the core task is to construct a neural distinguisher with high accuracy. Since the accuracy of a neural distinguisher represents its ability to discern the block cipher from the random permutation. In order to enhance the accuracy of neural distinguishers, cryptanalysts mainly focus on two aspects: neural network architecture and input data format.

In terms of neural network architecture, Bao et al. [10] compared three different network structures for Simon with block size 32 bits and key length 64 bits (Simon32/64) [7], including Residual Network (ResNet) [11], Dense Network (DenseNet) [12] and Squeeze-and-Excitation Network (SENet) [13]. Inspired by the structure of GoogLeNet [14], Zhang,

Wang, and Wang [15] and Zhang, Wang, and Chen [16] used multiple parallel convolutional layers to capture more dimensional information. Subsequently, Gohr, Leander, and Neumann [17] presented the effects of various hyperparameters on the accuracy of the neural distinguishers. In order to provide a generic training pipeline for differential cryptanalysis, Bellini et al. [18] used dilated convolution [19] to address the problem of near and long-range dependencies. In addition, they introduced an evolutionary algorithm to search for the candidate differences with high bias scores for constructing neural distinguishers. These two components together form an automated framework for training differential neural distinguishers.

In terms of input data format, Baksi [20] adopted the output differences of the t (≥ 2) input differences to train the differential neural distinguisher by simulating the all-in-one method in [21]. At Eurocrypt 2021, Benamira et al. [22] improved the accuracy of neural distinguishers by using batches of ciphertext inputs. Subsequently, Chen et al. [23] adopted multiple ciphertext pairs to effectively improve the accuracy of the neural distinguisher for Speck32/64 [7], Chaskey [24], Present64/80 [8], data encryption standard (DES) [25] and Secure Hash Algorithm 3-256 (SHA3-256) [26]. In addition, there are some studies to improve the accuracy of the neural distinguisher by providing the derivation information from the previous rounds, such as [10, 15, 16, 27].

So far, most studies of neural distinguishers have followed Gohr's [6] framework, which considers only a single input difference. In contrast, there are few studies considering multiple input differences, only [20, 28, 29]. However, it is worth noting that this approach has the potential to yield superior neural distinguishers. In this paper, we aim to study this further.

1.1. Our Contributions. First, we discuss the advantages of using two different differences to construct the neural distinguishers compared to using a single difference or more than two differences. Then, we provide an automated framework to build the neural distinguishers using two differences with four steps: difference selection, sample generation, training pipeline, and evaluation scheme. To validate the effectiveness of our approach, we apply it to four block ciphers: Simon, Speck, Simeck, and Hight, and achieve improved accuracy under the same conditions. Using the simple training pipeline, we can achieve the following enhancements compared with the results achieved by Bellini et al. [18].

- For Simon32/64 and Simon with block size 48 bits and key length 96 bits (Simon48/96), we improve the accuracy of their 11-round and 12-round neural distinguishers from 0.516 and 0.506 to 0.525 and 0.515, respectively. For Simon with block size 64 bits and key length 128 bits (Simon64/128), we not only obtain a superior 13-round neural distinguisher, but also achieve a 14-round neural distinguisher with the accuracy of 0.506.
- For Speck32/64 and Speck with block size 64 bits and key length 128 bits (Speck64/128), we enhance the accuracy of their 8-round neural distinguishers from 0.508 and 0.532 to 0.514 and 0.559, respectively. For Speck with block size 48 bits and key length 96 bits

(Speck48/96), we not only obtain a better 7-round neural distinguisher, but also an 8-round neural distinguisher with accuracy of 0.506.

- For Simeck with block size 32 bits and key length 64 bits (Simeck32/64), we increase the accuracy of its 11-round neural distinguisher from 0.516 to 0.525. For Simeck with block size 48 bits and key length 96 bits (Simeck48/96) and Simon64/128, we not only obtain improved 14-round or 17-round neural distinguishers, but also achieve the better neural distinguishers in the rounds 15 and 18.
- For Hight, we raise the accuracy of its 14-round related-key differential neural distinguisher from 0.563 to 0.640.

By employing the polishing step, we can further enhance the accuracy of our neural distinguishers. However, this process requires a lot of computational resources and time. Therefore, we only apply it to Simon32/64, Simeck32/64, and Speck32/64. For Simon32/64 and Simeck32/64, we not only achieve a neural distinguisher with higher accuracy for rounds 11, but also obtain a 12-round neural distinguisher with accuracy of 0.505. Even compared to the neural distinguishers using more complex data samples in [10, 15], our neural distinguishers is competitive. For Speck32/64, we achieve a neural distinguisher with an accuracy of 0.519, which is better than the accuracy of 0.514 achieved by existing neural distinguishers. Our neural distinguishers are also comparable when converted to the neural distinguishers using multiple ciphertext pairs, as shown in Table 1. All these results show the effectiveness of our method in improving the accuracy of neural distinguishers.

1.2. Organization. Section 2 provides some concepts and principles of (related-key) differential cryptanalysis and neural distinguishers. Section 3 details the automated construction process of our (related-key) differential neural distinguisher using two differences. The enhanced (related-key) differential neural distinguishers are constructed for Simon, Speck, Simeck, and Hight in Section 4. Section 5 concludes this paper.

2. Preliminaries

2.1. (Related-Key) Differential Cryptanalysis. Differential cryptanalysis is a chosen plaintext attack that focuses on the probabilistic propagation of plaintext differences during the encryption. In traditional differential cryptanalysis, the cryptographer focuses on the high-probability difference (α and β) in the single-key scenario, where α denotes the plaintext difference and β denotes the ciphertext difference. Suppose that $C = E_K(P)$ denotes encrypting the plaintext P with the encryption algorithm E and the master key K to obtain the ciphertext C , the differential probability of (α and β) is:

$$\frac{\#\{E_K(P) \oplus E_K(P \oplus \alpha) = \beta\}}{|P|}.$$

For the related-key scenario, it uses master keys that satisfy the difference δ to encrypt the plaintexts instead of the same keys. Thus, the differential probability of (α , δ , and β) becomes:

TABLE 1: Comparison of the accuracy for the neural distinguishers.

Cipher	Round	$k = 1$	$k = 2$	$k = 4$	$k = 8$	$k = 16$	$k = 32$	Source
Simon32/64	11	0.516	0.524	0.536	0.552	0.572	0.602	[18]
		0.517	—	—	—	—	—	[10] ^{1,2}
		—	0.524	0.539	0.559	0.588	—	[15] ^{1,2}
		0.518	0.527	0.540	0.558	0.581	0.615	[18] ¹
		0.520	0.529	0.543	0.563	0.589	0.625	[17] ¹
		0.525	0.536	0.551	0.574	0.604	0.642	Section 4.1
	12	0.529	0.542	0.560	0.586	0.620	0.668	Section 4.1 ¹
		—	—	—	0.515	0.523	—	[15] ^{1,2}
Simon48/96	12	0.505	0.508	0.512	0.518	0.526	0.537	Section 4.1 ¹
		0.506	0.508	0.513	0.517	0.523	0.528	[18]
Simon64/128	13	0.515	0.523	0.531	0.545	0.564	0.591	Section 4.1
		0.518	0.527	0.539	0.556	0.580	0.612	[18]
	14	0.526	0.540	0.558	0.584	0.618	0.668	Section 4.1
		0.506	0.509	0.513	0.520	0.528	0.539	Section 4.1
Speck32/64	8	0.508	0.514	0.518	0.528	0.540	0.556	[18]
		0.514	0.520	0.529	0.539	0.555	0.577	Section 4.2
		0.514	0.521	0.530	0.543	0.560	0.585	[17] ¹
		0.514	0.522	0.532	0.546	0.564	0.592	[18] ¹
		0.519	0.527	0.540	0.557	0.581	0.614	Section 4.2 ¹
		0.587	0.625	0.674	0.739	0.817	0.900	[18]
Speck48/96	7	0.620	0.671	0.737	0.818	0.902	0.966	Section 4.2
	8	0.506	0.511	0.515	0.521	0.529	0.541	Section 4.2
Speck64/128	8	0.532	0.550	0.571	0.599	0.637	0.686	[18]
		0.559	0.584	0.618	0.666	0.730	0.806	Section 4.2
Simeck32/64	11	—	0.502	—	—	—	—	[30]
		0.516	0.523	0.534	0.552	0.575	0.607	[18]
		—	—	—	0.565	—	—	[10] ²
		—	—	—	0.567	—	—	[31] ^{1,2}
		0.525	0.539	0.557	0.583	0.618	0.665	Section 4.3
		0.527	0.541	0.562	0.590	0.629	0.680	Section 4.3 ¹
	12	—	—	—	0.515	—	—	[10] ^{1,2}
		—	—	—	0.516	—	—	[31] ^{1,2}
		0.505	0.509	0.514	0.521	0.531	0.546	Section 4.3 ¹
		0.515	0.522	0.535	0.553	0.577	0.612	[18]
Simeck48/96	14	0.520	0.532	0.548	0.572	0.606	0.652	Section 4.3
	15	0.504	0.506	0.509	0.513	0.518	0.524	[18]
		0.505	0.509	0.512	0.519	0.529	0.541	Section 4.3
		0.514	0.522	0.534	0.552	0.578	0.614	[18]
Simeck64/128	17	—	0.529	0.547	0.570	0.608	0.657	Section 4.3
		0.504	0.506	0.510	0.517	0.525	0.537	[18]
	18	0.507	0.509	0.514	0.523	0.536	0.554	Section 4.3
		0.563	0.597	0.644	0.708	0.788	0.876	[18]
Hight	14	0.640	0.707	0.788	0.877	0.952	0.991	Section 4.4

Note: k , the number of ciphertext pairs.

Abbreviations: Simeck32/64, Simeck with block size 32 bits and key length 64 bits; Simeck48/96, Simeck with block size 48 bits and key length 96 bits; Simeck64/128, Simeck with block size 64 bits and key length 128 bits; Simon32/64, Simon with block size 32 bits and key length 64 bits; Simon48/96, Simon with block size 48 bits and key length 96 bits; Simon64/128, Simon with block size 64 bits and key length 128 bits; Speck32/64, Speck with block size 32 bits and key length 64 bits; Speck48/96, Speck with block size 48 bits and key length 96 bits; Speck64/128, Speck with block size 64 bits and key length 128 bits.

¹The neural distinguishers are trained with the staged training strategy or the polishing pipeline.

²The neural distinguishers employ a more complex data format than ciphertext pairs.

$$\frac{\#\{E_K(P) \oplus E_{K \oplus \delta}(P \oplus \alpha) = \beta\}}{|P| + |K|}.$$

2.2. Convolutional Neural Networks (CNNs). CNN is a feed-forward neural network with convolutional architecture. It is widely used in various fields, such as image recognition [32, 33], video analysis [34, 35], natural language processing [36, 37], and so on. A CNN usually comprises convolutional layers, pooling layers, and fully connected layers. Among them, the convolutional layer is used to extract features through convolution operation. The pooling layer performs down-sampling to reduce the parameters and computation of the neural network. The fully connected layer integrates the outputs of the convolutional and pooling layers for the final classification and prediction.

In this paper, we use not only standard convolutions but also dilated convolutions [19]. It introduces a parameter called dilation rate to define the spacing between the elements of the convolution kernel. By adjusting dilation rate, we can expand the receptive field of the kernel while maintaining a constant number of parameters. This approach can capture long-range dependencies in the data more effectively.

2.3. (Related-Key) Differential Neural Distinguisher. At CRYPTO 2019, Gohr [6] first introduced the real and random approach for training the neural distinguishers with ciphertext pairs. This is an important milestone in the application of deep learning techniques to differential cryptanalysis. Later, the differential neural distinguisher was extended to the related-key scenario by Wang, Wang, and He [28]. Since then, the (related-key) neural distinguisher has been widely used in different block ciphers, such as Simon [10, 18], Simeck [31, 38], Present [16, 18, 39], and others.

2.3.1. Sample Composition. In [6], Gohr's differential neural distinguishers were trained using two types of ciphertext pairs. One type of ciphertext pairs, labeled with 1, is obtained by encrypting the plaintext pairs with the difference α using the same master key. The other class of ciphertext pairs, labeled with 0, is obtained by encrypting the random plaintext pairs using the same master key. In the related-key scenario, these two types of ciphertext pairs are obtained by encrypting the above plaintext pairs using the master key pairs that satisfy the difference δ .

2.3.2. Difference Evaluation. At fast software encryption (FSE) 2024, Bellini et al. [18] showed that the weighted bias score of ciphertext differences can approximately represent the suitability of different differences for constructing (related-key) differential neural distinguishers. However, computing the exact weighted bias score is impractical because it requires enumerating all keys and plaintexts, but we can approximate it using t samples. The related definitions are as follows:

Definition 1 (Bias score). For the cipher primitive $E: \mathbb{F}_2^n \times \mathbb{F}_2^k \rightarrow \mathbb{F}_2^n$, the r -round bias score of the plaintext difference $\alpha \in \mathbb{F}_2^n$ and the key difference $\delta \in \mathbb{F}_2^k$, computed from t samples and denoted as $b_r^t(\alpha, \delta)$, is the sum of the biases for each bit position j in the output difference, that is,

$$\tilde{b}_r^t(\alpha, \delta) = \frac{1}{n} \sum_{j=1}^n \left(0.5 - \frac{1}{t} \sum_{i=1}^t (E_{K_i}(X_i) \oplus E_{K_i \oplus \delta}(X_i \oplus \alpha))_j \right).$$

Definition 2 (Weighted bias score). The R -round weighted bias score of the plaintext difference α and the key difference δ , denoted as $\tilde{W}_R^t(\alpha, \delta)$, is the sum of the products of the number of rounds in each round and its bias score.

$$\tilde{W}_R^t(\alpha, \delta) = \sum_{r=1}^R r \times \tilde{b}_r^t(\alpha, \delta).$$

2.3.3. Training Process. The complete training process of a (related-key) differential neural distinguisher consists of two parts: an online phase and an offline phase. The online phase focuses on training the (related-key) differential neural distinguisher, while the offline phase uses the trained (related-key) differential neural distinguisher to execute the attack. In the online phase, the plaintext difference and the key difference used to generate the samples should be selected first. For the single-key scenario, the key difference is 0. In this paper, we use the weighted bias scores of ciphertext difference to approximately evaluate different differences. Once the difference is chosen, we proceed to generate the training set and test set. Each set comprises half of the positive samples labeled as 1 and half of the negative samples labeled as 0. In [6, 18], the positive sample is the ciphertext pairs generated by the selected difference, while the negative sample is the ciphertext pairs generated by random difference. Subsequently, the training set is used to train the neural network model. If the accuracy of this model surpasses 0.5, it indicates that we successfully obtain a usable (related-key) differential neural distinguisher.

In the offline phase, the pretrained neural network model is used to distinguish the ciphertext pairs generated by the block cipher or random permutation. If over half of the ciphertext pairs score above 0.5, we deduce that they stem from the block cipher; otherwise, they originate from the random permutation.

3. Automated Construction of Our Neural Distinguishers

In this section, we first introduce the motivation to construct our neural distinguisher using two differences instead of just one or more. Subsequently, we give an automated process for constructing our neural distinguisher with four steps: difference selection, sample generation, training pipeline, and evaluation scheme.

3.1. Motivation. At Eurocrypt 2021, Benamira et al. [22] showed that Gohr's neural distinguisher in [6] can well identify ciphertext pairs that exhibit truncated differences in the last two rounds for Speck32/64, suggesting that it may acquire differential-linear characteristics. Subsequently, Gohr, Leander, and Neumann [17] further investigated the generality of this conclusion in various ciphers. Their study showed that the neural distinguisher can indeed learn features beyond the differences from the ciphertext pairs for Katan [40] and ChaCha [41].


```

1: initial_population ← [RandomInt(0,  $2^n - 1$ ) for 1024 times]
2: Sort initial_population by their bias score  $\tilde{b}_r^t(\cdot)$  (descending order)
3: current_population ← first 32 elements of initial_population
4: for iterations ← 0 to 50 do
5:   candidates ← []
6:   for  $i \leftarrow 0$  to 32 do
7:     for  $j \leftarrow i + 1$  to 32 do
8:       if RandomFloat(0, 1) < 0.1 then
9:          $m \leftarrow 1$ 
10:      else
11:         $m \leftarrow 0$ 
12:      end if
13:      Add current_population,  $\oplus$  current_population,  $\oplus$  ( $m = \text{RandomInt}(0, n - 1)$ ) to candidates
14:    end for
15:  end for
16:  Sort candidates by  $\tilde{b}_r^t(\cdot)$  (descending order)
17:  current_population ← first 32 elements of candidates
18: end for
19: return candidates

```

ALGORITHM 1: Evolutionary optimizer.

However, for Simon [7], Present [8], and Skinny [42], only the differences in ciphertext pairs provide useful information. In addition, their study highlights the close connection between the accuracy of the neural distinguisher and the mean absolute distance between the ciphertext difference distribution and the uniform distribution.

Based on these studies, we choose to build the differential neural distinguisher using the ciphertext pairs generated by two different nonzero input differences. The reason for choosing two differences, rather than one or more, is mainly due to the consideration of how the input differences propagate and how the output differences conflict. If only one input difference is selected, as the number of rounds increases, some output differences will tend to be uniformly distributed due to the confusion and diffusion properties of the block cipher. This poses a challenge for the neural network to distinguish them from uniformly distributed ciphertext pairs. However, if these uniformly distributed ciphertext pairs are generated by another, more favorable, input difference, the average absolute distance between their output differences will be more pronounced. This enhancement would allow the neural network to distinguish them more effectively, since both of them follow their own unique distributions.

There are two reasons for limiting the number of input differences to two rather than more: first, the input differences that retain their own unique distributions after several rounds are rare, and second, the more ciphertext pairs there are, the higher the possibility of collision.

3.2. Difference Selection. The scheme of Bellini et al. [18]. At FSE 2024, Bellini et al. [18] showed that the weighted bias score can approximately evaluate the suitability of different differences for constructing (related-key) differential neural distinguishers, as outlined in Section 2.3. Additionally, they

also propose a genetic algorithm to automatically generate suitable differences, as detailed in Algorithm 1. In this algorithm, the initial population consists of 1024 random input differences. They are then sorted in descending order according to their bias score $\tilde{b}_r^t(\cdot)$. The 32 best differences form the new population. Subsequently, there are 50 population iterations, each of which continuously generates new candidate differences by performing bit-wise XOR on two differences in the current population and flipping a random bit with probability 10%. Among these candidates, the 32 candidate differences with the highest bias scores go to the next iteration.

In order to enhance the algorithm's generality and prevent instances where certain differences exhibit low bias scores in the early rounds but high bias scores in later rounds, this genetic algorithm starts at round 1 and increases until the highest bias score is less than the threshold T_b of 0.05. We denote the maximum number of rounds at this time as R . The R -round weighted bias score is used as the final evaluation metric for all candidate differences. The differences with scores within 10% of the optimal bias score are taken as the final candidate differences for constructing the (related-key) differential neural distinguishers.

3.2.1. Our Scheme. However, our neural distinguisher requires two differences, so the approach outlined in [18] does not meet our requirements. Our analysis in Section 3.1 shows that these two differences should satisfy two basic conditions. First, they must exhibit favorable weighted bias scores to ensure that their ciphertext pairs contain unique and easily identifiable features. Second, maximize the difference between their ciphertext pairs as much as possible, thereby, enhancing the ability of the neural network to distinguish them.

TABLE 2: The relative weighted bias scores of various differences (α_0 , $\beta_0 = 0$ and $\alpha_1, \beta_1 = 0$) and the accuracy of their differential neural distinguishers for 8-round Speck32/64 in the single-key scenario.

α_1, α_0	0x400000		0x408000	
	Accuracy	Scores	Accuracy	Scores
0x502000	0.514	2.770	0.509	2.944
0x1408000	0.513	3.085	0.507	3.051
0x81000	0.513	2.880	0.510	2.891
0x200000	0.511	2.796	0.505	3.027
0x604000	0.510	2.903	0.502	2.912
0x8000	0.510	2.741	0.501	2.897
0x600000	0.510	2.797	0.506	3.031
0x408000	0.509	2.904	—	—
0x102000	0.508	2.832	0.512	2.876
0x400000	—	—	0.510	2.904

Although the method proposed in [18] is useful for selecting differences with high weighted bias scores, the solution for evaluating the discrepancy between the two differences is still lacking.

The most direct way is to construct neural distinguishers for all the combinations of differences with good weighted bias scores. But sometimes this requires a lot of time and computational resources due to the large number of combinations that need to be evaluated, especially for the ciphers with many candidate differences, such as Simon and Simeck. Therefore, in addition to traversing all the combinations of candidate differences, we provide a fast solution for difference selection.

Following the concept of weighted bias scores, we attempt to use the relative weighted bias scores of the two differences to approximate their availability. The relative weighted bias score is calculated by multiplying the number of rounds in each round and its relative bias scores, where the relative bias score of the differences ($\alpha_0, \delta_0, \alpha_1$, and δ_1) is the deviation between their bit bias scores as follow:

$$\begin{aligned} \tilde{b}_r^t(\alpha_0, \delta_0, \alpha_1, \delta_1) \\ = \frac{1}{n} \sum_{j=0}^{n-1} \left(\frac{1}{t} \sum_{i=0}^{t-1} (E_{K_i \oplus \delta_0}(X_i \oplus \alpha_0) - E_{K_i \oplus \delta_1}(X_i \oplus \alpha_1))_j \right). \end{aligned}$$

However, the results are not satisfactory. On the one hand, the relative weighted bias score and the accuracy of the neural distinguisher are not perfectly positively correlated. On the other hand, the scores of these combinations are very close. This can be observed from Table 2, which shows the relative weighted bias scores of various differences ($\alpha_0, \beta_0 = 0$ and $\alpha_1, \beta_1 = 0$) and the accuracy of their differential neural distinguishers for 8-round Speck32/64 in the single-key scenario. The α_0 is derived using the method in [18], and the β_1 is the 10 differences with the best weighted bias scores.

From the perspective of deep learning and data features, we can fix α_0 and δ_0 to be the optimal or top-ranked difference used to construct the neural distinguisher with one difference. Since these differences can generate the ciphertext

pairs that contain the richest data features, thus, providing more accurate and richer information to the neural network. At this time, we only need to filter a suitable α_1 and δ_1 from the remaining candidate differences to make the differential neural distinguisher achieve the best accuracy. To avoid the scarcity or absence of candidate differences for α_1 and δ_1 , we specify that there are at least 10 candidate differences. This ensures that we can build better neural distinguishers using two differences.

3.3. Samples Generation. Gohr's neural distinguisher encrypts the plaintext pairs that adhere to the specified difference or random differences to generate positive and negative ciphertext pairs with labels 1 or 0 as described in Section 2.3. In our approach, we utilize two differences to generate the positive and negative samples, respectively. The sample generation process is summarized below.

- Generate N plaintext pairs $(P_{i,0}, P_{i,1})$ and key pairs $(K_{i,0}, K_{i,1})$, $i \in [1, N]$. Half of them satisfy $P_{i,1} = P_{i,0} \oplus \alpha_0$ and $K_{i,1} = K_{i,0} \oplus \delta_0$, while the other half conform to $P_{i,1} = P_{i,0} \oplus \delta_1$ and $K_{i,1} = K_{i,0} \oplus \delta_1$.
- The encryption algorithm E is used to encrypt the plaintext pairs $(P_{i,0}, P_{i,1})$ with the key pairs $(K_{i,0}, K_{i,1})$ to obtain the ciphertext pairs $(C_{i,0}, C_{i,1})$, denoted as:

$$(C_{i,0}, C_{i,1}) = (E_{K_{i,0}}(P_{i,0}), E_{K_{i,1}}(P_{i,1})), i \in [1, N].$$

- The Y_i of a sample with plaintext difference α_j and key difference δ_j is set to j , where $j \in \{0, 1\}$. The logical expression is as follows:

$$Y_i = \begin{cases} 0, & \text{if } P_{i,0} \oplus P_{i,1} = \alpha_0 \text{ and } K_{i,0} \oplus K_{i,1} = \delta_0 \\ 1, & \text{if } P_{i,0} \oplus P_{i,1} = \alpha_1 \text{ and } K_{i,0} \oplus K_{i,1} = \delta_1 \end{cases}.$$

- The data format of the i -th sample is $((C_{i,0}, C_{i,1}), Y_i)$.

3.4. Training Pipeline. The choice of neural network architecture and hyperparameters plays a pivotal role in training neural distinguishers. In the pioneering work, Gohr [6] adopted the ResNet [11] for Speck32/64. Subsequently, this architecture and its variants have been widely used, such as [10, 20, 38]. However, these architectures are tailored for specific block ciphers, since designing a generalized neural network for various block ciphers is a difficult task. To address this problem, Gohr, Leander, and Neumann [17] introduced an automatic hyperparameter tuning method and applied it to six different block ciphers. However, on the one hand, this method does not work for all ciphers, such as advanced encryption standard (AES), as shown in [18], and on the other hand, it does not support neural distinguishers with the highest number of rounds, which would require a staged training approach. Therefore, Bellini et al. [18] used a generalized neural network called DBitNet to adapt different grouping ciphers. In addition, they

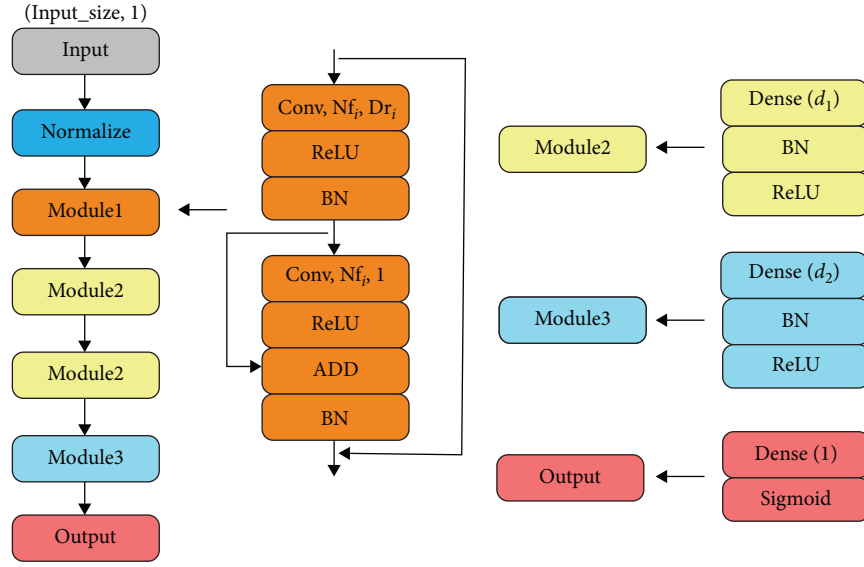


FIGURE 1: The network architecture of DBitNet. The input_size is twice the length of the ciphertext. The number of output filters Nf_i is $32 + i \times 16$ in the i -th dilated convolution. Dr_i is the dilation rate in the i -th dilated convolution, which is equal to $\text{input_size}/(i \times 2) - 1$ but with a minimum of 3. d_1 and d_2 represent the number of neurons in Module2 and Module3, where $d_1 = 256$ and $d_2 = 64$. ADD is a skip connection implemented through addition. BN stands for batch normalization. ReLU and Sigmoid are two different activation functions. ReLU, rectified linear unit.

provided a simple training pipeline to train the neural distinguisher with the maximum number of rounds and a polishing step to further enhance its accuracy by decreasing the learning rate of the Adam optimizer for three times, from 10^{-4} to 10^{-5} to 10^{-6} .

3.4.1. DBitNet. An overview of the DBitNet is shown in Figure 1. Its input is the binary representations of ciphertext pairs with the shape $(\text{input_size}, 1)$, where input_size is twice the length of the ciphertext. The input data is then normalized to the range $[-1, 1]$. The normalized output passes through one Module1, two Module2, one Module3, and then reaches the output layer.

Module1 integrates dilated convolution, batch normalization (BN), rectified linear unit (ReLU) activation functions, and skip connections through addition operations (ADD). In the first convolution layer, the dilation rate Dr_i is dynamically computed by repeatedly dividing the input_size by 2 and subtracting 1 until it reaches a minimum value of 3, that is, $Dr_i = \text{input_size} \div (i \times 2) - 1$, where i starts from 0 and increases until Dr_i equals 3. As the dilation rate Dr_i is continuously halved, the number of output filters Nf_i is gradually increased by 16 from the initial 32, i.e., $Nf_i = 32 + i \times 16$. This can capture varying degrees of long-range dependence. In the second convolution layer, the dilation rate is fixed to 1 to explore the dependencies between neighboring bits. Module2 contains a dense layer with $d_1 = 256$ neurons, a BN, and a ReLU activation function. Module3 has the same architecture as Module2 but contains $d_2 = 64$ neurons. The output layer is an output unit with the sigmoid activation function.

3.4.2. The Simple Training Pipeline. In this paper, we reuse the simple pipeline proposed by Bellini et al. in [18] to automate the training of our (related-key) neural distinguishers

with the maximum number of rounds instead of the manual staged training strategy in [6, 10, 15]. This simple training pipeline first trains a neural distinguisher directly from the first two rounds with weighted bias scores greater than T_b , and then, reuses the distinguisher from the previous round to train the distinguisher for the next round until the accuracy of the neural distinguisher is less than a threshold of 0.505. In addition, the Amsgrad algorithm [43] introduced at International Conference on Learning Representations (ICLR) 2018 is adopted to automatically adjust the learning rate to avoid manual work.

3.4.3. A Polishing Step. In most cases, the simple training pipeline is enough to generate competitive distinguishers with the highest number of round compared to the staged training strategy used in [6, 10, 15]. However, it is worth noting that the staged training strategy retrains the neural network with 10^9 training samples, while the simple training pipeline only uses 10^7 . As a result, the neural distinguisher obtained from the simple training pipeline may not outperform the neural distinguisher trained with the staged training strategy in the accuracy of the highest number of rounds. To solve this problem, a polishing step is introduced to further improve the accuracy of the neural distinguisher by utilizing 10^9 training samples. The neural distinguisher with the maximum number of rounds is retrained three times by using the Adam optimizer with learning rates of 10^{-4} , 10^{-5} , and 10^{-6} , respectively.

3.5. Evaluation Scheme. To fairly compare our neural distinguishers constructed using a single ciphertext pair with the neural distinguisher built using multiple ciphertext pairs, we adopt the scheme proposed by Gohr, Leander, and Neumann [17] to transform our neural distinguishers (using a single

ciphertext pair) into the ones that utilize an arbitrary number of ciphertext pairs. Specifically, if the probability of m samples $(p_1, \dots, p_m)$ is independent, the probability of m samples can be expressed as follows:

$$\frac{1}{1 + \prod_{i=1}^m \frac{1-p_i}{p_i}}.$$

4. Applications

In this section, we apply our scheme to four block ciphers: Simon [7], Speck [7], Simeck [44], and Hight [45]. Building on the research in [18], we construct the single-key differential neural distinguishers for Simon, Speck, and Simeck using our simple training pipeline with 10 epochs. For the Hight, we employ 40 epochs to build its related-key differential neural distinguisher. In addition, to demonstrate the effectiveness of the polishing step for our neural distinguishers, we apply it to Simon32/64, Speck32/64, and Simeck32/64. For all of them, we acquire the neural distinguishers with higher accuracy, as shown in Table 1.

4.1. Simon. Simon [7] is the lightweight block cipher proposed by the National Security Agency (NSA) in 2013. It adopts the classical Feistel structure, and its round function consists of bit-wise XOR, AND, and left circular shifts, delivering notable advantages in hardware implementation. In this paper, we consider three variants of Simon with block sizes of 32, 48, and 64 bits and key lengths of 64, 96, and 128 bits, known as Simon32/64, Simon48/96, and Simon64/128, respectively.

First, we apply the difference selection method in Section 3.2 to search for the differences with high weighted bias scores. The number of candidate differences that satisfy 0.1-close solutions is 16, 23, and 24, respectively. We then use these differences to construct the differential neural distinguishers that use one difference in [18]. The best differential neural distinguishers are constructed from the 0x400, 0x200000, and 0x8000, respectively. The maximum number of rounds for these neural distinguishers is 11, 12, and 13 with accuracies of 0.516, 0.506, and 0.518, respectively.

Subsequently, we fix α_0 as the best difference above and adopt the remaining candidate differences as α_1 to train our neural distinguishers using the simple training pipeline. For the 11-round Simon32/64, 12-round Simon48/96, and 13-round Simon64/128, our best neural distinguishers are constructed using $\alpha_1 = 0x100, 0x400000, \text{ and } 0x2000$ with improved accuracies of 0.525, 0.515, and 0.526, respectively. For the Simon64/128, we also gain the 14-round differential neural distinguisher with the accuracy of 0.506.

Furthermore, by employing the polishing step for Simon32/64, we successfully improve the accuracy of its 11-round differential neural distinguisher from 0.525 to 0.529 and achieve a 12-round neural distinguisher with an accuracy of 0.505 by using the difference pair ($\alpha_0 = 0x40$ and $\alpha_1 = 0x100$).

Finally, for a fair comparison between our neural distinguishers and known neural distinguishers using one or more ciphertext pairs, we transform our neural distinguishers into

the neural distinguishers using $k \in \{2, 4, 8, 16, 32\}$ ciphertext pairs based on the methods in Section 3.5, as shown in Table 1. Compared to the results in [10, 15, 18], our neural distinguishers exhibit higher accuracy.

4.2. Speck. Speck is a lightweight block cipher proposed with Simon. It employs a modified Feistel structure, and its round function consists of cyclic left and right shifts, bit-wise XOR, and modular addition 2^n operations, exhibiting superior software performance.

For the Speck cipher family [7], we also consider their variants with block sizes of 32, 48, and 64 bits, and key lengths of 64, 96, and 128 bits, denoted as Speck32/64, Speck48/96, and Speck64/128, respectively. Each of them has two candidate differences with $\varepsilon = 0.1$ for the α_0 . Since there are only two available candidate differences, we take the 10 differences with the most favorable weighted bias scores as candidates for α_1 . We then use the simple training pipeline to construct our neural distinguisher by traversing all combinations of α_0 and α_1 .

For Speck32/64 and Speck64/128, our 8-round best neural distinguishers are constructed from the (0x400000, 0x502000) and (0x808000000, 0x901000000) with accuracies of 0.514 and 0.559, respectively. For Speck48/96, we not only achieve a superior 7-round neural distinguisher with an accuracy 0.620 by using (0x80800000, 0x480800000), but also obtain an 8-round neural distinguisher with an accuracy of 0.506 by using (0x80800000, 0x90100000). In addition, for the Speck32/64, we can further improve the accuracy of its 8-round neural distinguisher from 0.514 to 0.519 through the polishing step. Compared to the accuracy of 0.514 in [10, 17], we achieve a better accuracy. The advantages of our neural distinguishers become more obvious when these results are extended to the neural distinguishers using $k \in \{2, 4, 8, 16, 32\}$ ciphertext pairs, as detailed in Table 1.

4.3. Simeck. Simeck is a lightweight block cipher proposed by Yang et al. [44] at International Conference on Cryptographic Hardware and Embedded Systems (CHES) 2015. It combines the design strengths of Simon and Speck. Its round function is very similar to Simon's, with the only difference being the choice of parameters for the circular shift. Its key expansion algorithm reuses its round function, similar to the block cipher Speck, and thus has superior performances in both software and hardware implementations.

For the Simeck cipher family [44], we also consider its variants with block sizes of 32, 48, and 64 bits and key lengths of 64, 96, and 128 bits, that is, the Simeck32/64, Simeck48/96, and Simeck64/128, respectively. They have 16, 20, and 24 candidate differences satisfying $\varepsilon = 0.1$. Their best neural distinguishers are constructed from 0x4000, 0x80000, and 0x2000000 with accuracies of 0.516, 0.515, and 0.514 in rounds 11, 14, and 17, respectively.

We then further construct our neural distinguishers through the simple training pipeline by fixing α_0 to 0x4000, 0x80000, and 0x2000000 and selecting α_1 from the remaining differences. For Simeck32/64, Simeck48/96, and Simeck64/128, the accuracies of our best neural distinguisher trained using $\alpha_1 = 0x8000, 0x10000, \text{ and } 0x20000$ in rounds

11, 14, and 17 are 0.525, 0.520, and 0.519, respectively. In addition, we also obtain the neural distinguishers for 15-round Simeck48/96 and 18-round Simeck64/128 using $\alpha_1 = 0x2000$ and $0x20000$ with accuracies of 0.505 and 0.507, respectively. Using the polishing step, we can further improve the accuracy of our 11-round neural distinguisher to 0.527 for Simeck32/64. The superior accuracy for our neural distinguishers with multiple ciphertext pairs is given in Table 1.

4.4. Hight. Hight [45] is a generalized Feistel-like block cipher with 64-bit block size and 128-bit key length. It only relies on some basic operations, such as bit-wise XOR, addition modulo 2^8 , and left rotation. As a result, it can provide a lightweight hardware implementation with low cost and low power consumption. Unlike most block ciphers, its key scheme also contains the whitened key besides the round key. In addition, there is an initial transformation and a final transformation before and after the round function. Unlike most block ciphers, its encryption scheme also requires a whitening key in addition to a round key for the initial transformation and the final transformation before and after the round function.

Bellini et al. [18] yielded a 10-round single-key differential neural distinguisher and a 14-round related-key differential neural distinguisher for Hight. The number of rounds in the related-key scenario is significantly higher than that in the single-key scenario, so we focus on building the related-key differential neural distinguisher for Hight. The optimal 14-round related-key differential neural distinguisher is constructed with the plaintext difference $0x80000000000000$ and key difference $0x800000$ with 40 epochs, so we set $(0x80000000000000, 0x800000)$ as α_0 and δ_0 to construct our related-key neural distinguisher using the same number of epochs.

Similar to the previous experiment, we first use Algorithm 1 to automatically generate the related-key candidate differences with high weighted bias scores, but the results are not satisfactory, which may be due to the large differential search space. The Algorithm 1 prefers the candidates with the plaintext difference of 0. Therefore, we choose α_1 and δ_1 from all the plaintext differences and key differences with hamming weight of 1. We first evaluate the weighted bias scores of their combinations and then choose the top 25 differences with the highest scores as the candidates for α_1 and δ_1 . Our best 14-round neural distinguisher is constructed from the $\alpha_1 = 0x40$ and $\delta_1 = 0x40000000000000000000000000000000$ with an accuracy of 0.640, while it is 0.563 in [18]. When comparing them in multiple ciphertext pairs, the advantage of our neural distinguishers is more prominent, as shown in Table 1.

5. Conclusion and Discussion

In this paper, we first discuss the limitation of neural distinguisher using only a single difference, and point out the advantages of constructing a neural distinguisher using two differences from the inherent principles of neural distinguishers and differential cryptanalysis. We then provide a generic training pipeline for our neural distinguishers, which

consists of four steps: difference selection, sample generation, training pipeline, and evaluation scheme. Finally, to demonstrate the effectiveness of our method, we apply it to four different block ciphers: Simon, Speck, Simeck, and Hight, and obtain improved accuracy over existing results.

For the single-key scenario, the framework of Bellini et al. [18] that we used can automatically construct neural distinguishers. But for the related-key differential neural distinguisher, we need to manually filter the plaintext difference and key difference, since Algorithm 1 cannot automatically find the satisfactory results in the huge combination space of plaintext difference and key difference. We will further investigate this problem in the future.

Data Availability Statement

The data that support the findings of this study are openly available at <https://github.com/differentialedistinguisher/AutoND-New>.

Conflicts of Interest

The authors declare no conflicts of interest.

Funding

This work is supported by the National Key Research and Development Program of China (Grant 2022YFB2701900), the National Natural Science Foundation of China (Grants 62472172 and 62072181), the Open Fund of Advanced Cryptography and System Security Key Laboratory of Sichuan Province (Grant SKLACSS-202301), and Shanghai Trusted Industry Internet Software Collaborative Innovation Center.

References

- [1] E. Biham and A. Shamir, "Differential Cryptanalysis of DES-Like Cryptosystems," *Journal of Cryptology* 4, no. 1 (1991): 3–72.
- [2] M. Matsui, "On Correlation Between the Order of S-Boxes and the Strength of DES," in *Advances in Cryptology—EUROCRYPT'94*, vol. 950 of *Lecture Notes in Computer Science*, (Springer, Berlin, Heidelberg, 1994): 366–375.
- [3] S. Sun, L. Hu, P. Wang, K. Qiao, X. Ma, and L. Song, "Automatic Security Evaluation and (Related-key) Differential Characteristic Search: Application to SIMON, PRESENT, LBlock, DES(L) and Other Bit-Oriented Block Ciphers," in *Advances in Cryptology—ASIACRYPT 2014*, vol. 8873 of *Lecture Notes in Computer Science*, (Springer, Berlin, Heidelberg, 2014): 158–178.
- [4] D. Gerault, M. Minier, and C. Solnon, "Constraint Programming Models for Chosen Key Differential Cryptanalysis," in *Principles and Practice of Constraint Programming*, vol. 9892 of *Lecture Notes in Computer Science*, (Springer, Cham, 2016): 584–601.
- [5] Z. Liu, Y. Li, and M. Wang, "Optimal Differential Trails in SIMON-like Ciphers," *IACR Transactions on Symmetric Cryptology* 2017 (2017): 358–379.
- [6] A. Gohr, "Improving Attacks on Round-Reduced Speck32/64 Using Deep Learning," in *Advances in Cryptology—CRYPTO 2019*, vol. 11693 of *Lecture Notes in Computer Science*, (Springer, Cham, 2019): 150–179.
- [7] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, "The SIMON and SPECK Lightweight Block

- Ciphers,” in *Proceedings of the 52nd Annual Design Automation Conference*, (ACM, 2015): 1–6.
- [8] A. Bogdanov, L. R. Knudsen, G. Leander, et al., “PRESENT: An Ultra-Lightweight Block Cipher,” in *Cryptographic Hardware and Embedded Systems-CHES 2007*, vol. 4727 of *Lecture Notes in Computer Science*, (Springer, Berlin, Heidelberg, 2007): 450–466.
 - [9] D. J. Bernstein, S. Kölbl, S. Lucks, et al., “Gimli: A Cross-Platform Permutation,” in *Cryptographic Hardware and Embedded Systems-CHES 2017*, vol. 10529 of *Lecture Notes in Computer Science*, (Springer, Cham, 2017): 299–320.
 - [10] Z. Bao, J. Guo, M. Liu, L. Ma, and Y. Tu, “Enhancing Differential-Neural Cryptanalysis,” in *Advances in Cryptology-ASIACRYPT 2022*, vol. 13791 of *Lecture Notes in Computer Science*, (Springer, Cham, 2022): 318–347.
 - [11] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, (Las Vegas, NV, USA: IEEE, 2016), 770–778.
 - [12] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, “Densely Connected Convolutional Networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, (CVF, 2017): 4700–4708.
 - [13] J. Hu, L. Shen, and G. Sun, “Squeeze-and-Excitation Networks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, (CVF, 2018): 7132–7141.
 - [14] C. Szegedy, W. Liu, Y. Jia, et al., “Going Deeper With Convolutions,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, (CVF, 2015): 1–9.
 - [15] L. Zhang, Z. Wang, and B. Wang, “Improving Differential-Neural Cryptanalysis With Inception,” *Cryptology ePrint Archive*, 2022.
 - [16] L. Zhang, Z. Wang, and Y. Chen, “Improving the Accuracy of Differential-Neural Distinguisher for DES, Chaskey, and PRESENT,” *IEICE Transactions on Information and Systems* E106.D, no. 7 (2023): 1240–1243.
 - [17] A. Gohr, G. Leander, and B. Neumann, “An Assessment of Differential-Neural Distinguishers,” *Cryptology ePrint Archive*, 2022.
 - [18] E. Bellini, D. Gerault, A. Hambitzer, and M. Rossi, “A Cipher-Agnostic Neural Training Pipeline With Automated Finding of Good Input Differences,” *Cryptology ePrint Archive*, 2022.
 - [19] F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” *arXiv preprint arXiv: 1511.07122*, 2015.
 - [20] A. Baksi, “Machine Learning-Assisted Differential Distinguishers for Lightweight Ciphers,” in *Classical and Physical Security of Symmetric Key Cryptographic Algorithms*, Computer Architecture and Design Methodologies, (Springer, Singapore, 2022): 141–162.
 - [21] M. R. Albrecht and G. Leander, “An All-In-One Approach to Differential Cryptanalysis for Small Block Ciphers,” in *Selected Areas in Cryptography*, vol. 7707 of *Lecture Notes in Computer Science*, (Springer, Berlin, Heidelberg, 2013): 1–15.
 - [22] A. Benamira, D. Gerault, T. Peyrin, and Q. Q. Tan, “A Deeper Look at Machine Learning-Based Cryptanalysis,” in *Advances in Cryptology-EUROCRYPT 2021*, vol. 12696 of *Lecture Notes in Computer Science*, (Springer, Cham, 2021): 805–835.
 - [23] Y. Chen, Y. Shen, H. Yu, and S. Yuan, “A New Neural Distinguisher Considering Features Derived From Multiple Ciphertext Pairs,” *The Computer Journal* 66, no. 6 (2023): 1419–1433.
 - [24] N. Mouha, B. Mennink, A. Van Herrewwege, et al., “Chaskey: An Efficient MAC Algorithm for 32-bit Microcontrollers,” in *Selected Areas in Cryptography-SAC 2014*, vol. 8781 of *Lecture Notes in Computer Science*, (Springer, Cham, 2014): 306–323.
 - [25] DE Standard, “Data Encryption Standard,” in *Federal Information Processing Standards Publication*, 112, (De Standaard, 1999).
 - [26] G. Bertoni, J. Daemen, M. Peeters, and G. Van Assche, “Keccak,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, (Berlin, Heidelberg: Springer, 2013), 313–314.
 - [27] J. S. Liu, J. J. Ren, S. Z. Chen, and M. M. Li, “Improved Neural Distinguishers With Multi-Round and Multi-Splicing Construction,” *Journal of Information Security and Applications* 74 (2023): 103461.
 - [28] G. Wang, G. Wang, and Y. He, “Improved Machine Learning Assisted (Related-key) Differential Distinguishers For Lightweight Ciphers,” in *2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, (Shenyang, China: IEEE, 2021), 164–171.
 - [29] H.-C. Su, X.-Y. Zhu, and D. Ming, “Polytopic Attack on Round-Reduced Simon32/64 Using Deep Learning,” in *Information Security and Cryptology*, vol. 12612 of *Lecture Notes in Computer Science*, (Springer, Cham, 2020): 3–20.
 - [30] H. Wang, J. Tian, X. Zhang, Y. Wei, and H. Jiang, “Multiple Differential Distinguisher of SIMECK32/64 Based on Deep Learning,” *Security and Communication Networks* 2022 (2022): 7564678.
 - [31] L. Zhang, J. Lu, Z. Wang, and C. Li, “Improved Differential-Neural Cryptanalysis for Round-Reduced SIMECK32/64,” *Frontiers of Computer Science* 17 (2023): 176817.
 - [32] R. Chauhan, K. K. Ghanshala, and R. Joshi, “Convolutional Neural Network (CNN) for Image Detection and Recognition,” in *2018 First International Conference on Secure Cyber Computing and Communication (ICSCCC)*, (Jalandhar, India: IEEE, 2018), 278–282.
 - [33] S. Hijazi, R. Kumar, and C. Rowen, “Using Convolutional Neural Networks for Image Recognition,” *Cadence Design Systems Inc.*: San Jose, CA, USA, vol. 9, no. 1, 2015.
 - [34] A. Ullah, J. Ahmad, K. Muhammad, M. Sajjad, and S. W. Baik, “Action Recognition in Video Sequences Using Deep Bi-Directional LSTM With CNN Features,” *IEEE Access* 6 (2018): 1155–1166.
 - [35] Z. Xu, Y. Yang, and A. G. Hauptmann, “A Discriminative CNN Video Representation for Event Detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, (CVF, 2015): 1798–1807.
 - [36] W. Yin, K. Kann, M. Yu, and H. Schütze, “Comparative Study of Cnn and Rnn for Natural Language Processing,” *arXiv preprint arXiv: 1702.01923*, 2017.
 - [37] H. Li, “Deep Learning for Natural Language Processing: Advantages and Challenges,” *National Science Review* 5, no. 1 (2018): 24–26.
 - [38] J. Lu, G. Liu, B. Sun, C. Li, and L. Liu, “Improved (Related-Key) Differential-Based Neural Distinguishers for SIMON and SIMECK Block Ciphers,” *The Computer Journal* 67, no. 2 (2024): 537–547.
 - [39] A. Jain, V. Kohli, and G. Mishra, “Deep Learning Based Differential Distinguisher for Lightweight Cipher Present,” *Cryptology ePrint Archive*, 2020.
 - [40] C. De Canniere, O. Dunkelman, and M. Knežević, “KATAN and Ktantan—a Family of Small and Efficient Hardware-Oriented Block Ciphers,” in *Cryptographic Hardware and Embedded Systems - CHES 2009*, vol. 5747 of *Lecture Notes in Computer Science*, (Springer, Cham, 2009): 272–288.

- [41] D. J. Bernstein, "Bernstein and Others, ChaCha, A Variant of salsa20," Workshop record of sasc, 2008, vol. 8, pp. 3–5, 2008.
- [42] C. Beierle, J. Jean, S. Kölbl, et al., "The SKINNY Family of Block Ciphers and Its Low-Latency Variant MANTIS," in *Advances in Cryptology—CRYPTO 2016*, Lecture Notes in Computer Science, (Springer, Berlin, Heidelberg, 2016): 123–153.
- [43] S. J. Reddi, S. Kale, and S. Kumar, "On the Convergence of Adam and Beyond," arXiv preprint arXiv: 1904.09237, 2019.
- [44] G. Yang, B. Zhu, V. Suder, M. D. Aagaard, and G. Gong, "The Simeck Family of Lightweight Block Ciphers," in *Cryptographic Hardware and Embedded Systems—CHES 2015*, vol. 9293 of *Lecture Notes in Computer Science*, (Springer, Berlin, Heidelberg, 2015): 307–329.
- [45] D. Hong, J. Sung, S. Hong, et al., "A New Block Cipher Suitable for Low-Resource Device," in *Cryptographic Hardware and Embedded Systems—CHES 2006*, vol. 4249 of *Lecture Notes in Computer Science*, (Springer, Berlin, Heidelberg, 2006): 46–59.